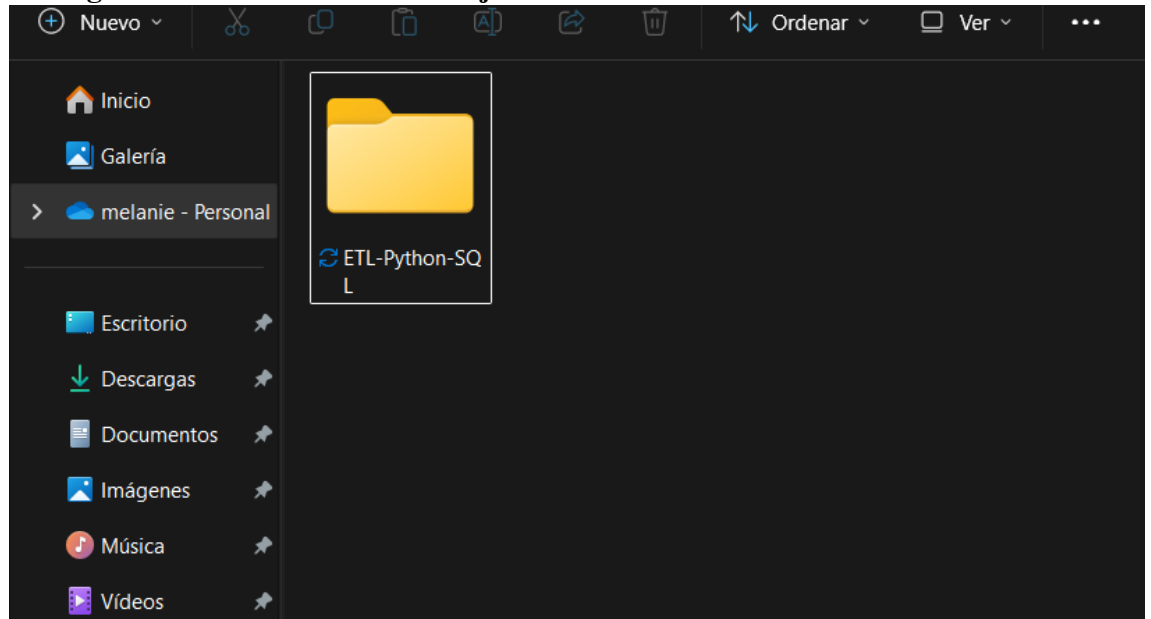
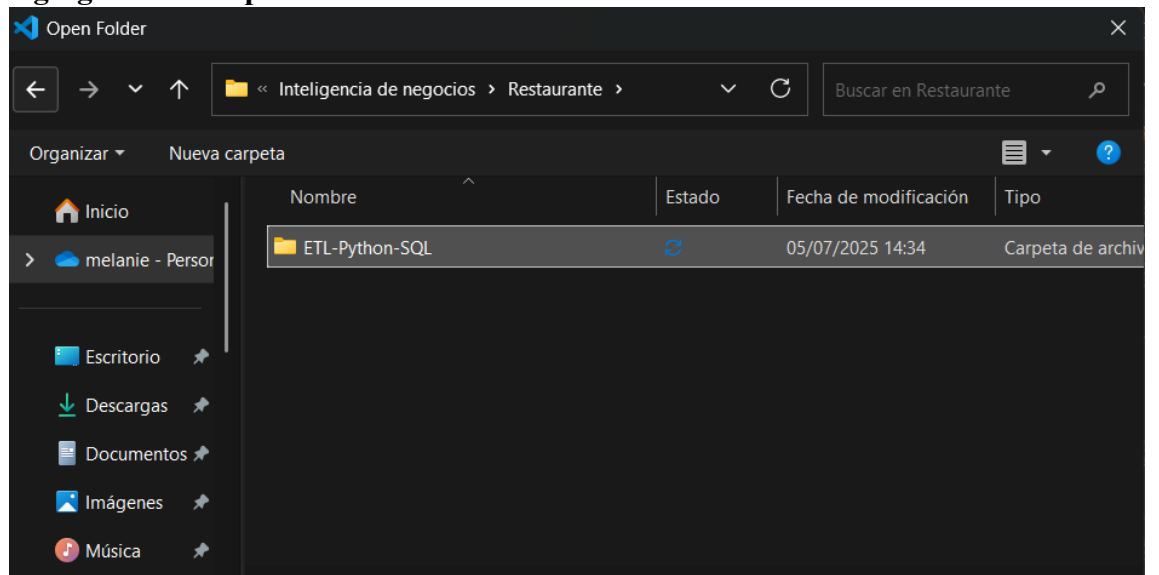


ETL Python para RestauranteMart (SQL Server)

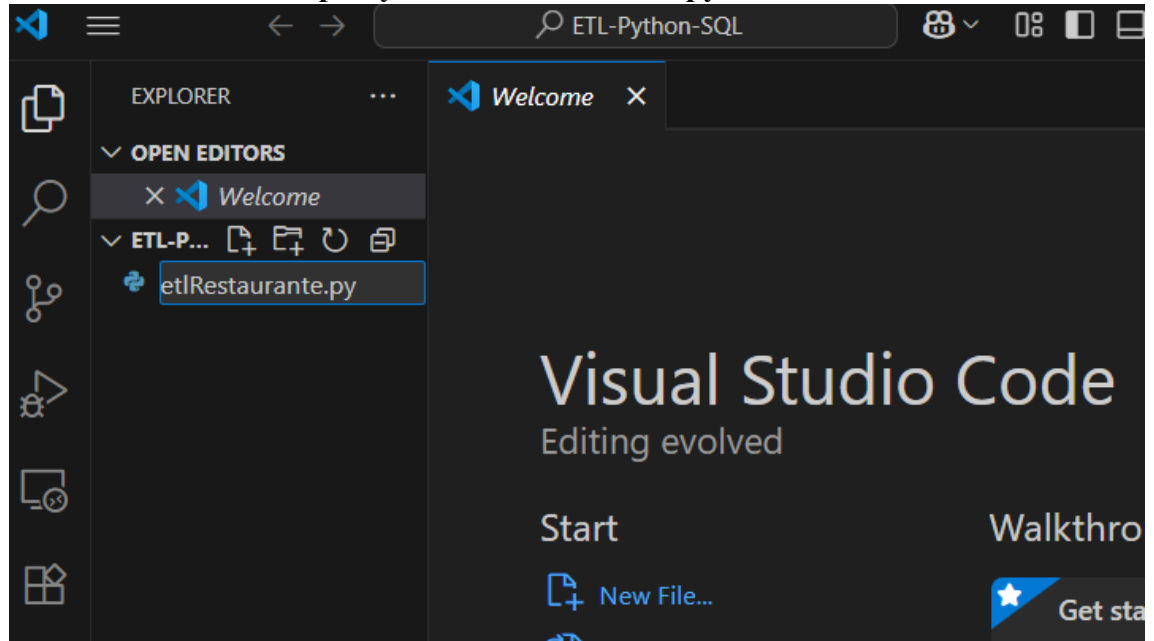
1. Configurar un directorio de Trabajo



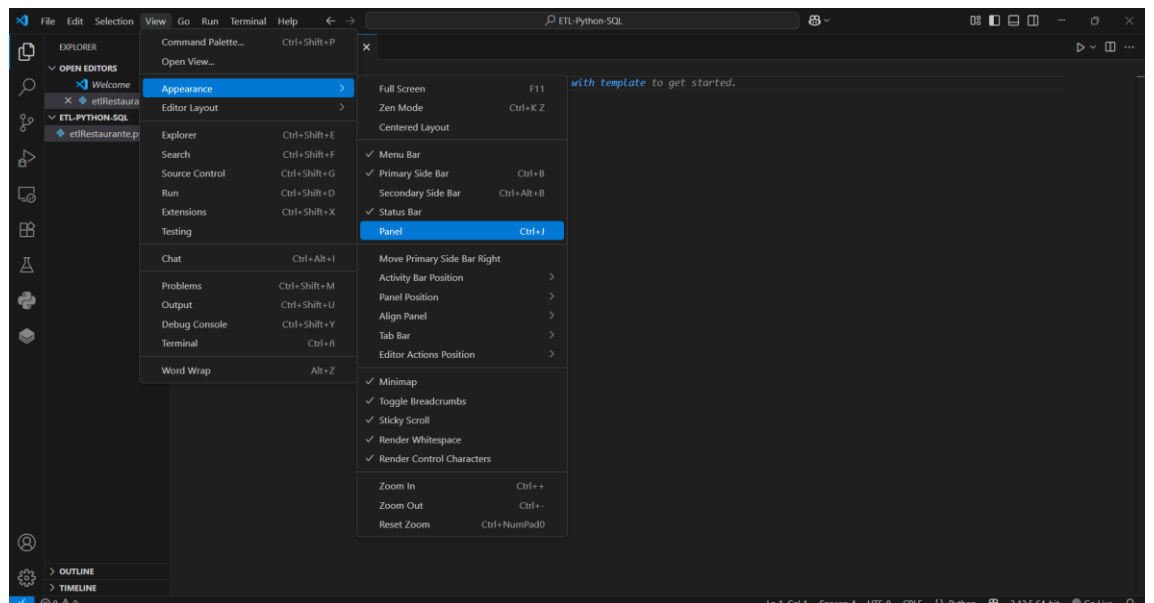
2. Agregar a Workspace



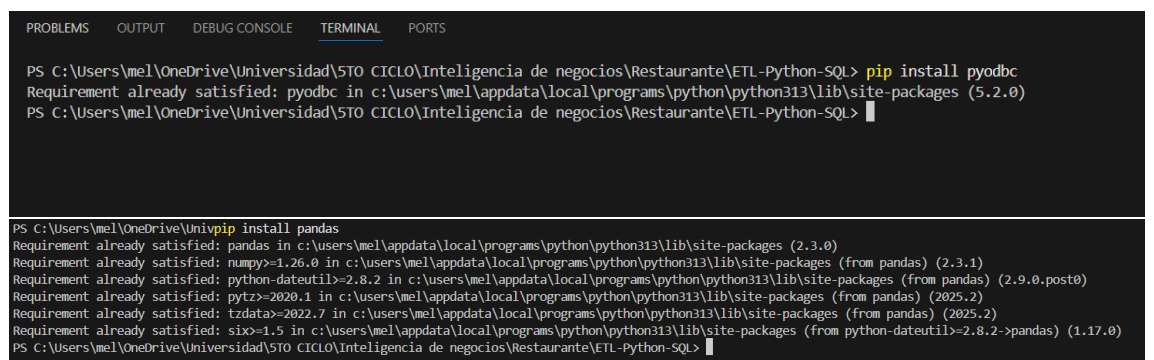
3. Crear nuevo archivo tipo Python: etlRestaurante.py



4. Abrir Panel o Terminal



5. Instalar en Terminal las librerías pyodbc y pandas (actualmente ya instaladas)



6. Codificar en Visual Studio Code

6.1. Importación de librerías

```
import pyodbc # Librería para conectar Python con bases de
datos SQL Server
import pandas as pd # Librería para manipular datos en
forma de tablas (DataFrames)
```

6.2. Conexión a base de datos transaccional (RestauranteDBasePrueba5m)

Se conecta a la base de datos original (transaccional), donde están los datos en bruto como clientes, empleados, productos, etc.

```
# ----- CONEXIÓN A BASE DE DATOS TRANSACCIONAL -----
print("----- CONEXIÓN A BASE DE DATOS TRANSACCIONAL -----")

# Conexión a la base de datos transaccional donde están los
datos operativos
try:
    conn_nw = pyodbc.connect('DRIVER={SQL
Server};SERVER=IDEAPAD\SQL;DATABASE=RestauranteDBasePrueba5
m;UID=sa;PWD=gnab')
    cursor_nw = conn_nw.cursor()
    print('Conexión a RestauranteDBasePrueba5m correcta.')
except Exception as e:
    print('Conexión a RestauranteDBasePrueba5m fallida:',
e)
    exit()
```

6.3. Consultas SQL para extraer datos

query1 hasta query5: consultas para crear las dimensiones Cliente, Mesa, Empleado, Producto y Tiempo.

query: consulta que genera la tabla de hechos (ventas) con ingresos, costos y márgenes.

```
# Consulta para obtener nombres completos de los clientes
query1 = """
SELECT
    (Nombres + ' ' + ApellidoPaterno + ' ' +
ApellidoMaterno) AS nombres
FROM CLIENTE.Cliente
ORDER BY 1;
"""
```

...

6.4. Ejecución de consultas y carga a DataFrames

Cada resultado de consulta se carga en una tabla en memoria usando pandas. Para transformarlos fácilmente antes de enviarlos al Data Warehouse.

```
df1 = pd.read_sql(query1, conn_nw)
df2 = pd.read_sql(query2, conn_nw)
df3 = pd.read_sql(query3, conn_nw)
df4 = pd.read_sql(query4, conn_nw)
df5 = pd.read_sql(query5, conn_nw)
df = pd.read_sql(query, conn_nw)
```

6.5. Transformación: convertir fechas

Convierte las fechas a un tipo reconocido por Python para trabajar con ellas más fácilmente.

```
print("----- TRANSFORMACIÓN DE DATOS -----")
df5['fecha'] = pd.to_datetime(df5['fecha']) # Convierte
texto a fecha real
```

6.6. Limpieza: eliminar duplicados

Se limpian las tablas para que no se inserten datos duplicados en el Data Warehouse.

```
dcliente = df1[['nombres']].drop_duplicates()
dmesa = df2[['ubicacion', 'capacidad', 'estado']]
dpleado = df3[['nombre',
'fechaContratacion']].drop_duplicates()
dproducto = df4[['categoria', 'tipo', 'precio',
'nombreProducto']].drop_duplicates()
dtiempo = df5[['fecha', 'anio', 'trimestre',
'nombreTrimestre', 'mes', 'nombreMes', 'semanaAnio',
'diaAnio', 'diaMes', 'diaSemana',
'nombreDia']].drop_duplicates()
```

6.7. Conexión al Data Warehouse (RestauranteMart)

Se conecta al Data Warehouse, que es el destino donde se cargarán los datos ya transformados.

```
try:
    conn_dw = pyodbc.connect('DRIVER={SQL
Server};SERVER=IDEAPAD\SQL;DATABASE=RestauranteMart;UID=sa;
PWD=gnab')
    cursor_dw = conn_dw.cursor()
    print('Conexión a RestauranteMart correcta.')
except Exception as e:
```

```
print('Conexión a RestauranteMart fallida:', e)
exit()
```

6.8. Limpieza previa de tablas en el DW

Elimina todos los datos actuales del DW y reinicia los contadores de claves primarias (IDENTITY) para empezar desde cero.

```
cursor_dw.execute("DELETE FROM dbo.FactVentas;")
tablas = [ 'Cliente', 'Mesa', 'Empleado', 'Producto',
'Tiempo']
for tabla in tablas:
    cursor_dw.execute(f"DELETE FROM DIM.{tabla};")
    cursor_dw.execute(f"DBCC CHECKIDENT ('DIM.{tabla}',
RESEED, 0);")
    conn_dw.commit()
cursor_dw.execute("DBCC CHECKIDENT ('dbo.FactVentas',
RESEED, 0);")
```

6.9. Carga de datos en tablas de dimensiones

Inserta fila por fila en las tablas de dimensiones: Cliente, Mesa, Empleado, Producto y Tiempo.

```
# Cargar datos a tabla DIM.Cliente
for _, row in dcliente.iterrows():
    cursor_dw.execute("INSERT INTO DIM.Cliente (nombres)
VALUES (?)", row.nombres)
    conn_dw.commit()
print("Carga de dimensión Cliente completada")
```

6.10. Carga de tabla de hechos (FactVentas)

Inserta los registros de ventas, que enlazan claves de dimensión y métricas como ingresos, costos, número de ventas, etc.

```
for _, row in df.iterrows():
    cursor_dw.execute("""
        INSERT INTO dbo.FactVentas (
            clientekey, mesakey, empleadokey, productokey,
            tiempokey, ingresosVenta, numeroVentas,
            ventaCompletas, costo, margen
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        row.clientekey, row.mesakey, row.empleadokey,
        row.productokey,
        row.tiempokey, row.ingresosVenta, row.numeroVentas,
        row.ventaCompletas, row.costo, row.margen)
    conn_dw.commit()
print('Carga de FactVentas completada.')
```

6.11. Cierre de conexiones

```
cursor_dw.close()
conn_dw.close()
conn_nw.close()
```

7. Código completo

```
import pyodbc # Librería para conectar Python con bases de
datos SQL Server
import pandas as pd # Librería para manipular datos en forma
de tablas (DataFrames)

print("Loading Session .....")

# ----- CONEXIÓN A BASE DE DATOS TRANSACCIONAL -----
print("----- CONEXIÓN A BASE DE DATOS TRANSACCIONAL -----
--")

# Conexión a la base de datos transaccional donde están los
datos operativos
try:
    conn_nw = pyodbc.connect('DRIVER={SQL
Server};SERVER=IDEAPAD\SQL;DATABASE=RestauranteDBasePrueba5m;U
ID=sa;PWD=gnaB')
    cursor_nw = conn_nw.cursor()
    print('Conexión a RestauranteDBasePrueba5m correcta.')
except Exception as e:
    print('Conexión a RestauranteDBasePrueba5m fallida:', e)
    exit()

# ----- EXTRACCIÓN DE DATOS -----
print("----- EXTRACCIÓN DE DATOS -----")

# Consulta para obtener nombres completos de los clientes
query1 = """
SELECT
    (Nombres + ' ' + ApellidoPaterno + ' ' + ApellidoMaterno)
AS nombres
FROM CLIENTE.Cliente
ORDER BY 1;
"""

# Consulta para obtener información de las mesas del
restaurante
query2 = """
SELECT
```

```

        Ubicacion AS ubicacion,
        Capacidad AS capacidad,
        Estado AS estado
FROM GENERAL.Mesa;
"""

# Consulta para obtener empleados y su fecha de contratación
query3 = """
SELECT
    NombreCompleto AS nombre,
    FechaContratacion AS fechaContratacion
FROM PERSONAL.Empleado
"""

# Consulta para obtener productos con su categoría, tipo y
precio
query4 = """
SELECT
    c.Nombre AS categoria,
    p.EsPreparado AS tipo,
    p.Precio AS precio,
    p.Nombre AS nombreProducto
FROM GENERAL.Producto p
INNER JOIN GENERAL.Categoria c ON p.Id_Categoria =
c.Id_Categoria
"""

# Consulta para construir La dimensión Tiempo a partir de Las
fechas de Los pedidos
query5 = """
SELECT DISTINCT
    Fecha AS fecha,
    YEAR(Fecha) AS anio,
    DATEPART(QUARTER, Fecha) AS trimestre,
    CASE DATEPART(QUARTER, Fecha)
        WHEN 1 THEN 'Q1'
        WHEN 2 THEN 'Q2'
        WHEN 3 THEN 'Q3'
        WHEN 4 THEN 'Q4'
    END AS nombreTrimestre,
    MONTH(Fecha) AS mes,
    DATENAME(MONTH, Fecha) AS nombreMes,
    DATEPART(WEEK, Fecha) AS semanaAnio,
    DATEPART(DAYOFYEAR, Fecha) AS diaAnio,
    DATEPART(DAY, Fecha) AS diaMes,
    DATEPART(WEEKDAY, Fecha) AS diaSemana,
    DATENAME(WEEKDAY, Fecha) AS nombreDia
FROM TRANSACCION.Pedido
ORDER BY 1

```

```

"""

# Consulta para construir la tabla de hechos FactVentas con
# todos los cálculos necesarios
query = """
-- Se crea una CTE para calcular el costo de los productos
según si son preparados o no
WITH ProductoCosto AS (
    SELECT
        p.Id_Producto AS productoid,
        p.EsPreparado,
        ISNULL(
            SUM(CASE
                WHEN p.EsPreparado = 1 THEN
ISNULL(inv_ing.CostoPorUnidad, 0) * ISNULL(pg_ing.Cantidad, 0)
            ELSE 0
            END),
            0
        ) AS CostoCalculadoIngredientes,
        ISNULL(
            MAX(CASE
                WHEN p.EsPreparado = 0 THEN
ISNULL(inv_direct.CostoPorUnidad, 0)
            ELSE 0
            END),
            0
        ) AS CostoDirectoProducto
    FROM [RestauranteDBasePrueba5m].[GENERAL].[Producto] p
    LEFT JOIN
    [RestauranteDBasePrueba5m].[GENERAL].[ProductoIngrediente]
pg_ing ON p.Id_Producto = pg_ing.Id_Producto
    LEFT JOIN
    [RestauranteDBasePrueba5m].[INVENTARIO].[Inventario] inv_ing
ON pg_ing.Id_Item = inv_ing.Id_Item
    LEFT JOIN
    [RestauranteDBasePrueba5m].[INVENTARIO].[Inventario]
inv_direct ON p.Nombre = inv_direct.ItemNombre
    GROUP BY p.Id_Producto, p.EsPreparado
)
-- Consulta principal con joins hacia dimensiones y cálculos
de ingresos, costos y márgenes
SELECT
    dc.clientekey,
    dm.mesakey,
    de.empleadokey,
    dp_dim.productokey,
    dt.tiempokey,
    p.Precio * dp.Cantidad AS ingresosVenta,
    dp.Cantidad AS numeroVentas,

```



```

CASE WHEN pe.Estado = 'Completado' THEN 1 ELSE 0 END AS
ventaCompletas,
dp.Cantidad * (
CASE
WHEN pc.EsPreparado = 1 THEN
pc.CostoCalculadoIngredientes
WHEN pc.EsPreparado = 0 THEN
pc.CostoDirectoProducto
ELSE 0
END
) AS costo,
(p.Precio * dp.Cantidad) - (
dp.Cantidad * (
CASE
WHEN pc.EsPreparado = 1 THEN
pc.CostoCalculadoIngredientes
WHEN pc.EsPreparado = 0 THEN
pc.CostoDirectoProducto
ELSE 0
END
)
) AS margen
FROM [RestauranteDBasePrueba5m].[TRANSACCION].[DetallePedido]
dp
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Producto] p
ON dp.Id_Producto = p.Id_Producto
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Categoria]
cat ON p.Id_Categoria = cat.Id_Categoria
INNER JOIN [RestauranteDBasePrueba5m].[TRANSACCION].[Pedido]
pe ON dp.Id_Pedido = pe.Id_Pedido
LEFT JOIN ProductoCosto pc ON dp.Id_Producto = pc.productoid
INNER JOIN [RestauranteDBasePrueba5m].[CLIENTE].[Cliente] c ON
c.Id_Cliente = pe.Id_Cliente
INNER JOIN [RestauranteDBasePrueba5m].[PERSONAL].[Empleado] e
ON e.Id_Empleado = pe.Id_Empleado
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Mesa] m ON
m.Id_Mesa = pe.Id_Mesa
INNER JOIN RestauranteMart.DIM.Cliente dc ON (c.Nombres + ' '
+ c.ApellidoPaterno + ' ' + c.ApellidoMaterno) = dc.nombres
INNER JOIN RestauranteMart.DIM.Mesa dm ON m.Ubicacion =
dm.ubicacion AND m.Capacidad = dm.capacidad AND m.Estado =
dm.estado
INNER JOIN RestauranteMart.DIM.Empleado de ON e.NombreCompleto
= de.nombre
INNER JOIN RestauranteMart.DIM.Producto dp_dim ON p.Nombre =
dp_dim.nombreProducto AND cat.Nombre = dp_dim.categoria AND
p.EsPreparado = dp_dim.tipo AND p.Precio = dp_dim.precio
INNER JOIN RestauranteMart.DIM.Tiempo dt ON CAST(pe.Fecha AS
DATE) = CAST(dt.fecha AS DATE)

```

```

ORDER BY dt.tiempokey;
"""

# ----- EJECUCIÓN DE CONSULTAS Y CARGA A DATAFRAMES -----
--
#Ejecutar cada una de las consultas SQL definidas previamente
y almacenar los resultados en objetos DataFrame de pandas
df1 = pd.read_sql(query1, conn_nw)
df2 = pd.read_sql(query2, conn_nw)
df3 = pd.read_sql(query3, conn_nw)
df4 = pd.read_sql(query4, conn_nw)
df5 = pd.read_sql(query5, conn_nw)
df = pd.read_sql(query, conn_nw)

# ----- TRANSFORMACIÓN DE DATOS -----
print("----- TRANSFORMACIÓN DE DATOS -----")
df5['fecha'] = pd.to_datetime(df5['fecha']) # Convierte texto
a fecha real

# ----- LIMPIEZA Y PREPARACIÓN -----
print("----- LIMPIEZA DE DATOS -----")
# Elimina duplicados en cada dimensión
dcliente = df1[['nombres']].drop_duplicates()
dmesa = df2[['ubicacion', 'capacidad', 'estado']]
dempleado = df3[['nombre',
'fechaContratacion']].drop_duplicates()
dproducto = df4[['categoria', 'tipo', 'precio',
'nombreProducto']].drop_duplicates()
dtiempo = df5[['fecha', 'anio', 'trimestre',
'nombreTrimestre', 'mes', 'nombreMes', 'semanaAnio',
'diaAnio', 'diaMes', 'diaSemana',
'nombreDia']].drop_duplicates()

# ----- CONEXIÓN A DATA WAREHOUSE -----
print("----- CONEXIÓN A BASE DE DATOS DIMENSIONAL -----")
try:
    conn_dw = pyodbc.connect('DRIVER={SQL
Server};SERVER=IDEAPAD\SQL;DATABASE=RestauranteMart;UID=sa;PWD
=gnab')
    cursor_dw = conn_dw.cursor()
    print('Conexión a RestauranteMart correcta.')
except Exception as e:
    print('Conexión a RestauranteMart fallida:', e)
    exit()

# ----- LIMPIEZA DE DIMENSIONES -----
print("----- LIMPIEZA DE DIMENSIONES -----")
# Borra las tablas y reinicia los contadores de IDENTITY

```

```

cursor_dw.execute("DELETE FROM dbo.FactVentas;")
tablas = [ 'Cliente', 'Mesa', 'Empleado', 'Producto',
'Tiempo']
for tabla in tablas:
    cursor_dw.execute(f"DELETE FROM DIM.{tabla};")
    cursor_dw.execute(f"DBCC CHECKIDENT ('DIM.{tabla}',
RESEED, 0);")
    conn_dw.commit()
cursor_dw.execute("DBCC CHECKIDENT ('dbo.FactVentas', RESEED,
0);")

# ----- CARGA DE DATOS A DIMENSIONES -----
print("----- CARGA DE DATOS A DIMENSIONES -----")

# Cargar datos a tabla DIM.Cliente
for _, row in dcliente.iterrows():
    cursor_dw.execute("INSERT INTO DIM.Cliente (nombres)
VALUES (?)", row.nombres)
conn_dw.commit()
print("Carga de dimensión Cliente completada")

# Cargar datos a DIM.Mesa
for _, row in dmesa.iterrows():
    cursor_dw.execute("INSERT INTO DIM.Mesa (ubicacion,
capacidad, estado) VALUES (?, ?, ?)", row.ubicacion,
row.capacidad, row.estado)
conn_dw.commit()
print("Carga de dimensión Mesa completada")

# Cargar datos a DIM.Empleado
for _, row in dempleado.iterrows():
    cursor_dw.execute("INSERT INTO DIM.Empleado(nombre,
fechaContratacion) VALUES (?, ?)", row.nombre,
row.fechaContratacion)
conn_dw.commit()
print("Carga de dimensión Empleado completada")

# Cargar datos a DIM.Producto
for _, row in dproducto.iterrows():
    cursor_dw.execute("INSERT INTO DIM.Producto (categoria,
tipo, precio, nombreProducto) VALUES (?, ?, ?, ?)",
row.categoria, row.tipo, row.precio, row.nombreProducto)
conn_dw.commit()
print("Carga de dimensión Producto completada")

# Cargar datos a DIM.Tiempo
for _, row in dtiempo.iterrows():
    cursor_dw.execute("""

```

```

        INSERT INTO DIM.Tiempo (fecha, anio, trimestre,
nombreTrimestre, mes, nombreMes, semanaAnio, diaAnio, diaMes,
diaSemana, nombreDia )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        row.fecha, row.anio, row.trimestre,
row.nombreTrimestre, row.mes, row.nombreMes, row.semanaAnio,
row.diaAnio, row.diaMes, row.diaSemana, row.nombreDia)
conn_dw.commit()
print("Carga de dimensión Tiempo completada")

# ----- CARGA DE TABLA DE HECHOS -----
for _, row in df.iterrows():
    cursor_dw.execute("""
        INSERT INTO dbo.FactVentas (
            clientekey, mesakey, empleadokey, productokey,
            tiempokey, ingresosVenta, numeroVentas,
            ventaCompletas, costo, margen
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        row.clientekey, row.mesakey, row.empleadokey,
row.productokey,
        row.tiempokey, row.ingresosVenta, row.numeroVentas,
row.ventaCompletas, row.costo, row.margen)
conn_dw.commit()
print('Carga de FactVentas completada.')

# ----- CIERRE DE CONEXIONES -----
print("----- EXTRACCION DE DATOS -----")
print("-----")
cursor_dw.close()
conn_dw.close()
conn_nw.close()
print('Conexiones cerradas correctamente.')

```

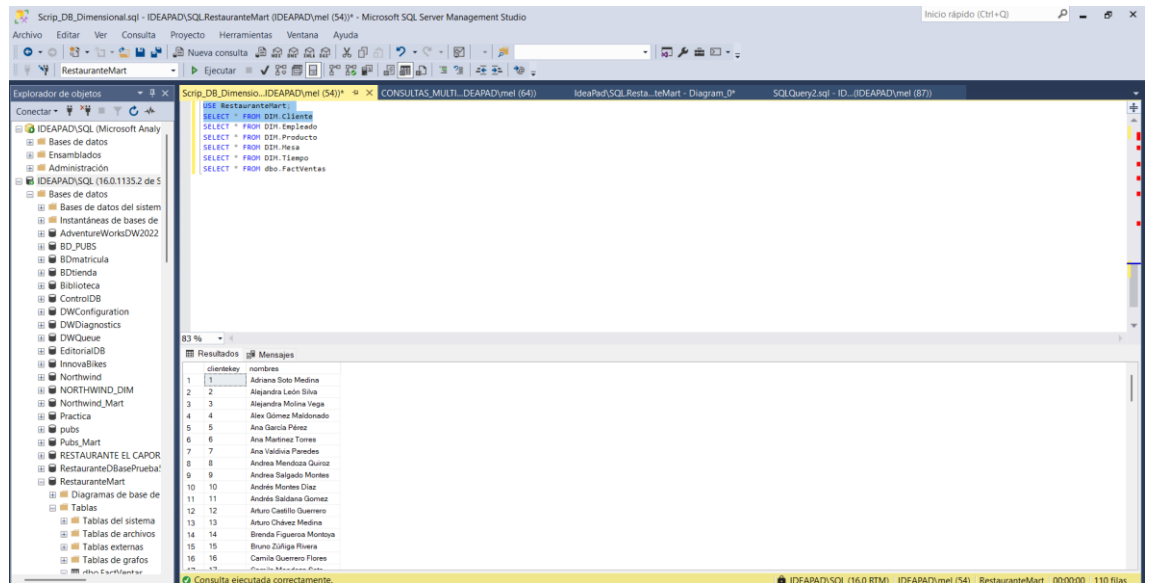
8. Resultado en Terminal

```

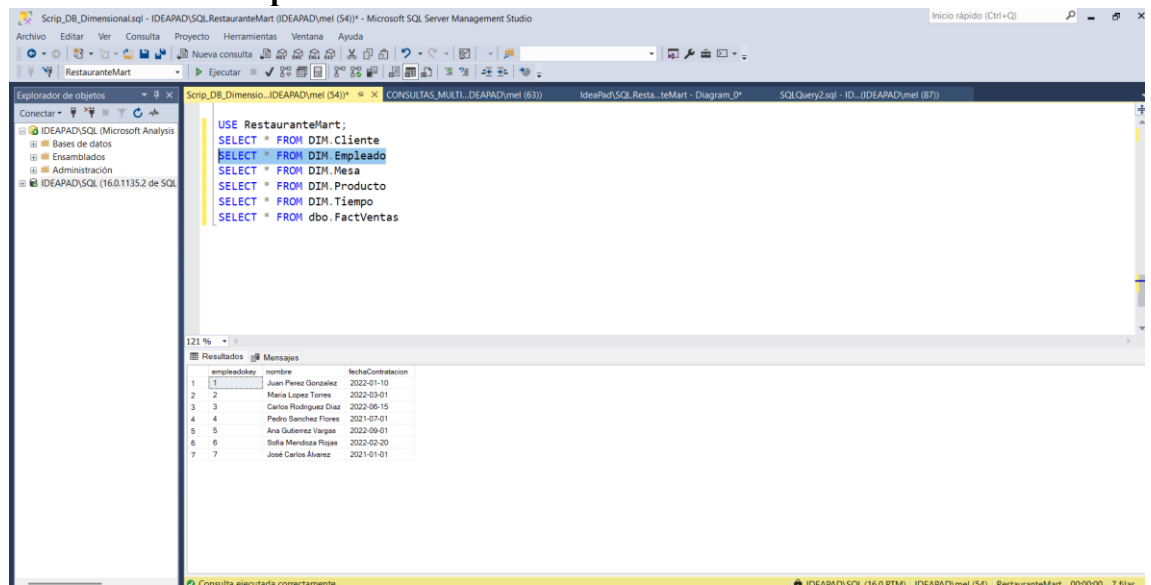
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
----- LIMPIEZA DE DIMENSIONES -----
----- CARGA DE DATOS A DIMENSIONES -----
Carga de dimensión Cliente completada
Carga de dimensión Mesa completada
Carga de dimensión Empleado completada
Carga de dimensión Producto completada
Carga de dimensión Tiempo completada
Carga de FactVentas completada.
----- EXTRACCION DE DATOS -----
-----
Conexiones cerradas correctamente.
PS C:\Users\mel\OneDrive\Universidad\5TO CICLO\Inteligencia de negocios\Restaurante\ETL-Python-SQL>

```

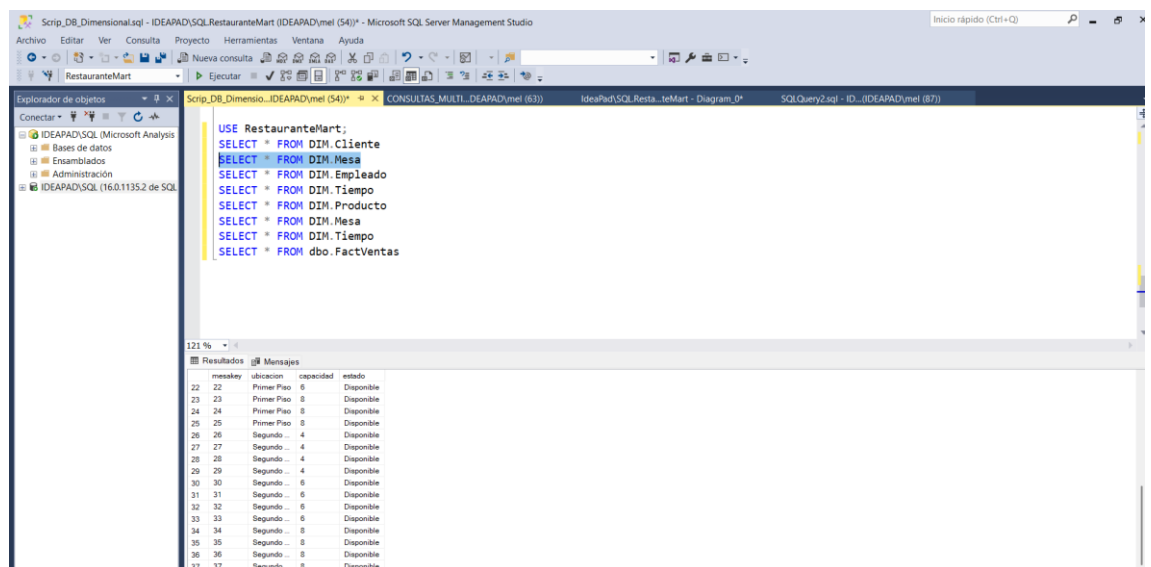
9. Poblando DIM.Cliente



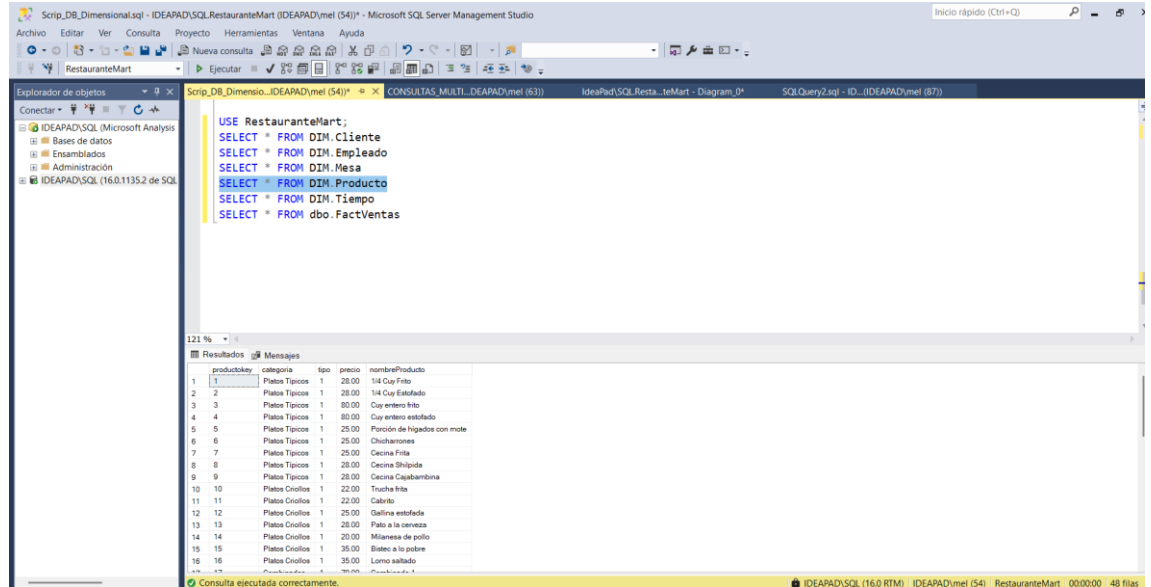
10. Poblando DIM.Empleado



11. Poblando DIM.Mesa



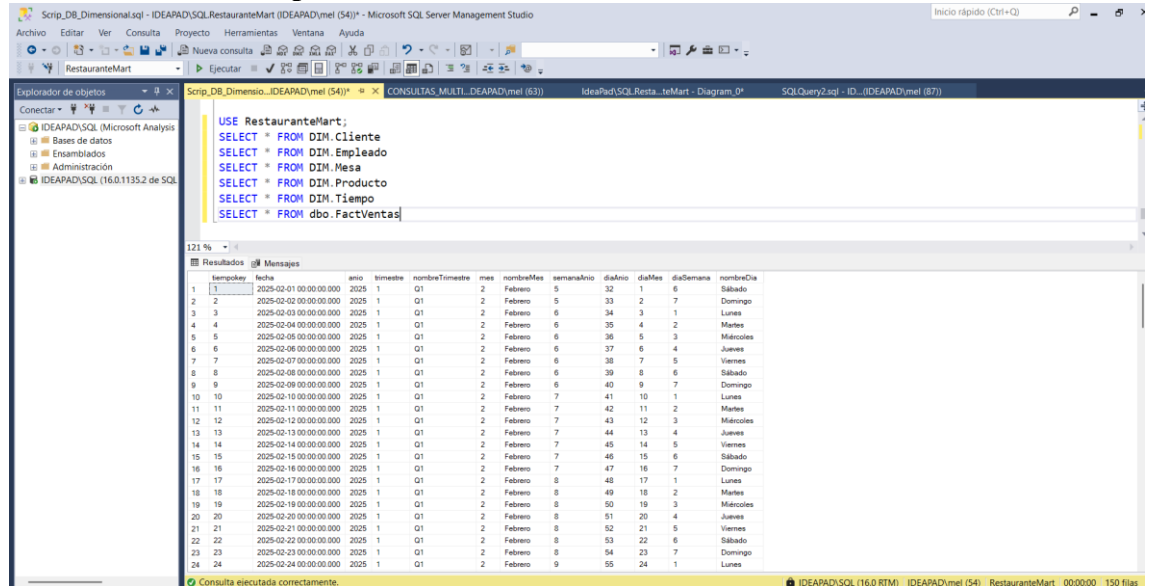
12. Poblando DIM.Producto



```
USE RestauranteMart;
SELECT * FROM DIM.Cliente
SELECT * FROM DIM.Empleado
SELECT * FROM DIM.Mesa
SELECT * FROM DIM.Producto
SELECT * FROM DIM.Tiempo
SELECT * FROM dbo.FactVentas
```

productokey	categoria	tipo	precio	nombreProducto
1	Platos Tipicos	1	28.00	1/4 Cuy Frito
2	Platos Tipicos	1	28.00	1/4 Cuy Entado
3	Platos Tipicos	1	80.00	Cuy entero frito
4	Platos Tipicos	1	80.00	Cuy entero asado
5	Platos Tipicos	1	25.00	Porción de Hígados con mote
6	Platos Tipicos	1	25.00	Chicharrones
7	Platos Tipicos	1	25.00	Cecina Frita
8	Platos Tipicos	1	28.00	Cecina Shishula
9	Platos Tipicos	1	28.00	Cecina Cajabambina
10	Platos Crofios	1	22.00	Truche frito
11	Platos Crofios	1	22.00	Cabruto
12	Platos Crofios	1	25.00	Galina asada
13	Platos Crofios	1	28.00	Pato a la cerveza
14	Platos Crofios	1	20.00	Milanesa de pollo
15	Platos Crofios	1	35.00	Bistec a lo pobre
16	Platos Crofios	1	35.00	Lomo salado

13. Poblando DIM.Tiempo



```
USE RestauranteMart;
SELECT * FROM DIM.Cliente
SELECT * FROM DIM.Empleado
SELECT * FROM DIM.Mesa
SELECT * FROM DIM.Producto
SELECT * FROM DIM.Tiempo
SELECT * FROM dbo.FactVentas
```

tiempokey	fecha	ano	trimestre	nombreTrimestre	mes	nombreMes	semanaAño	diaAño	diaSemana	nombreDia
1	2025-02-01 00:00:00.000	2025	1	Q1	2	Febrero	5	32	1	Sábado
2	2025-02-02 00:00:00.000	2025	1	Q1	2	Febrero	5	33	2	Domingo
3	2025-02-03 00:00:00.000	2025	1	Q1	2	Febrero	6	34	3	Lunes
4	2025-02-04 00:00:00.000	2025	1	Q1	2	Febrero	6	35	4	Martes
5	2025-02-05 00:00:00.000	2025	1	Q1	2	Febrero	6	36	5	Miércoles
6	2025-02-06 00:00:00.000	2025	1	Q1	2	Febrero	6	37	6	Jueves
7	2025-02-07 00:00:00.000	2025	1	Q1	2	Febrero	6	38	7	Viernes
8	2025-02-08 00:00:00.000	2025	1	Q1	2	Febrero	6	39	8	Sábado
9	2025-02-09 00:00:00.000	2025	1	Q1	2	Febrero	6	40	9	Domingo
10	2025-02-10 00:00:00.000	2025	1	Q1	2	Febrero	7	41	10	Lunes
11	2025-02-11 00:00:00.000	2025	1	Q1	2	Febrero	7	42	11	Martes
12	2025-02-12 00:00:00.000	2025	1	Q1	2	Febrero	7	43	12	Miércoles
13	2025-02-13 00:00:00.000	2025	1	Q1	2	Febrero	7	44	13	Jueves
14	2025-02-14 00:00:00.000	2025	1	Q1	2	Febrero	7	45	14	Viernes
15	2025-02-15 00:00:00.000	2025	1	Q1	2	Febrero	7	46	15	Sábado
16	2025-02-16 00:00:00.000	2025	1	Q1	2	Febrero	7	47	16	Domingo
17	2025-02-17 00:00:00.000	2025	1	Q1	2	Febrero	8	48	17	Lunes
18	2025-02-18 00:00:00.000	2025	1	Q1	2	Febrero	8	49	18	Martes
19	2025-02-19 00:00:00.000	2025	1	Q1	2	Febrero	8	50	19	Miércoles
20	2025-02-20 00:00:00.000	2025	1	Q1	2	Febrero	8	51	20	Jueves
21	2025-02-21 00:00:00.000	2025	1	Q1	2	Febrero	8	52	21	Viernes
22	2025-02-22 00:00:00.000	2025	1	Q1	2	Febrero	8	53	22	Sábado
23	2025-02-23 00:00:00.000	2025	1	Q1	2	Febrero	8	54	23	Domingo
24	2025-02-24 00:00:00.000	2025	1	Q1	2	Febrero	9	55	24	Lunes

14. Poblando Tabla FactVentas

Scrip_DB_Dimensional.sql - IDEAPAD\SQLRestaurantMart (IDEAPAD\met (54)) - Microsoft SQL Server Management Studio

Inicio rápido (Ctrl+Q)

ArchivoEditarVerConsultaProyectoHerramientasVentanaAyuda

Nueva consultaEjecutar

RestaurantMart

Explorador de objetos

Conectar

IDEAPAD\SQL (Microsoft Analysis)

Bases de datos

Ensamblados

Administración

IDEAPAD\SQL (16.0.1135.2 de SQL

Scrip_DB_Dimensio...IDEAPAD\met (54))

CONSULTAS.MULTI...DEAPAD\met (53))

ideaPad\SQL Resta...teMart - Diagram_0*

SQLQuery2.sql - ID... (IDEAPAD\met (87))

USE RestaurantMart;
SELECT * FROM DIM.Cliente
SELECT * FROM DIM.Empleado
SELECT * FROM DIM.Mesa
SELECT * FROM DIM.Producto
SELECT * FROM DIM.Tiempo
SELECT * FROM dbo.FactVentas

121 %

Resultados Mensajes

ventaKey	clienteKey	mesaKey	empleadoKey	productoKey	tiempoKey	ingreso/venta	numero/ventas	ventaCompleta	costo	margin
92_9206	79	8	2	5	143	50.00	2	1	6.50	43.50
92_9207	79	9	2	5	143	50.00	2	1	6.50	43.50
92_9208	79	10	2	5	143	50.00	2	1	6.50	43.50
92_9209	79	11	2	5	143	50.00	2	1	6.50	43.50
92_9210	79	12	2	5	143	50.00	2	1	6.50	43.50
92_9211	79	1	2	35	143	20.00	1	1	15_	5.00
92_9212	79	2	2	35	143	20.00	1	1	15_	5.00
92_9213	79	3	2	35	143	20.00	1	1	15_	5.00
92_9214	79	4	2	35	143	20.00	1	1	15_	5.00
92_9215	79	5	2	35	143	20.00	1	1	15_	5.00
92_9216	79	6	2	35	143	20.00	1	1	15_	5.00
92_9217	79	13	2	35	143	20.00	1	1	15_	5.00
92_9218	79	14	2	35	143	20.00	1	1	15_	5.00
92_9219	79	15	2	35	143	20.00	1	1	15_	5.00
92_9220	79	7	2	35	143	20.00	1	1	15_	5.00
92_9221	79	8	2	35	143	20.00	1	1	15_	5.00
92_9222	79	9	2	35	143	20.00	1	1	15_	5.00
92_9223	79	10	2	35	143	20.00	1	1	15_	5.00
92_9224	79	11	2	35	143	20.00	1	1	15_	5.00
92_9225	79	12	2	35	143	20.00	1	1	15_	5.00
92_9226	79	1	2	48	143	95.00	1	1	23_	71.85
92_9227	79	2	2	48	143	95.00	1	1	23_	71.85
92_9228	79	3	2	48	143	95.00	1	1	23_	71.85
92_9229	79	4	2	48	143	95.00	1	1	23_	71.85

Consulta ejecutada correctamente.

IDEAPAD\SQL (16.0 RTM) IDEAPAD\met (54) RestaurantMart 00:00:00 9,510 filas